

Algorithms Project

Longest Balanced Substring

Non-Recursive & Recursive Algorithms

Problem Statement

A string is balanced if it consists of exactly two different characters and both appear exactly the same number of times. A substring is a sequence of consecutive characters. Goal: find the length of the longest balanced substring in string S.

Examples:

"aabbab" → balanced (a appears 3 times, b appears 3 times)

"aabba" → NOT balanced (a appears 3 times, b appears 2 times)

"aabbcc" → NOT balanced (contains 3 distinct characters)

Task: Given a string S, find the length of the longest balanced substring.

Test Cases:

S = "cabbacc" → 4 ("abba" is the longest balanced substring)

S = "abababa" → 6 ("ababab" and "bababa" are both longest)

S = "aaaaaaa" → 0 (no balanced substring exists)

First Algorithm — Non-Recursive

1.1 Non – Recursive Pseudo Code

```
1. Algorithm GetLongestBalancedSubstring(s)
2. // text is a string
3. {
4.  n ← length(s); longestBalancedSubstring ← 0;

5.  for start ← 0 to n-1 do {
6.    c1 ← 0; c2 ← 0; countC1 ← 0; countC2 ← 0;

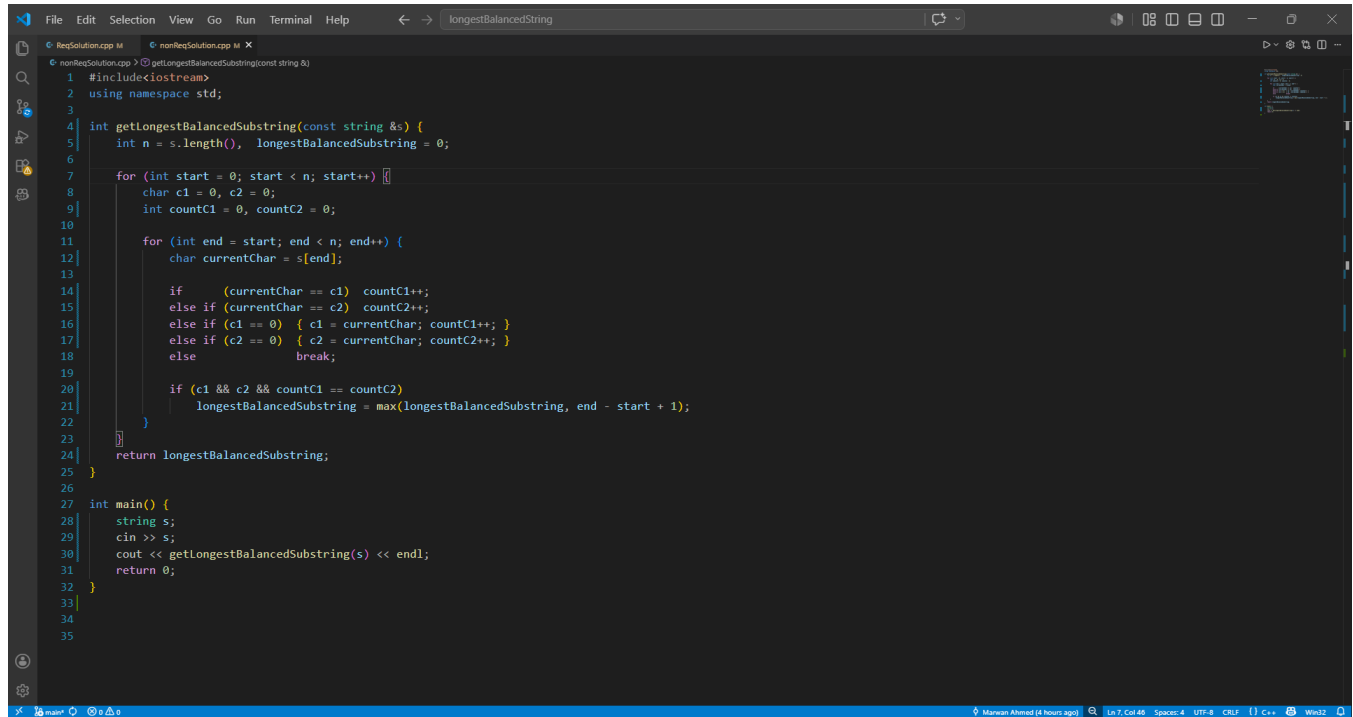
7.    for end ← start to n-1 do {
8.      currentChar ← s[end];

9.      if currentChar = c1 then countC1 ← countC1 + 1;
10.     else if currentChar = c2 then countC2 ← countC2 + 1;
11.     else if c1 = 0 then { c1 ← currentChar; countC1 ← 1; }
12.     else if c2 = 0 then { c2 ← currentChar; countC2 ← 1; }
13.     else break;

14.     if (c1 ≠ 0 AND c2 ≠ 0 AND countC1 = countC2) then
15.       longestBalancedSubstring ← max(longestBalancedSubstring, end - start + 1);
16.     }
17.  }

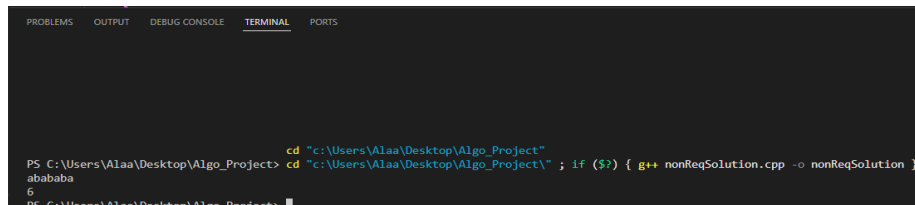
18. return longestBalancedSubstring;
19. }
```

1.2 Non – Recursive Implementation (C++)

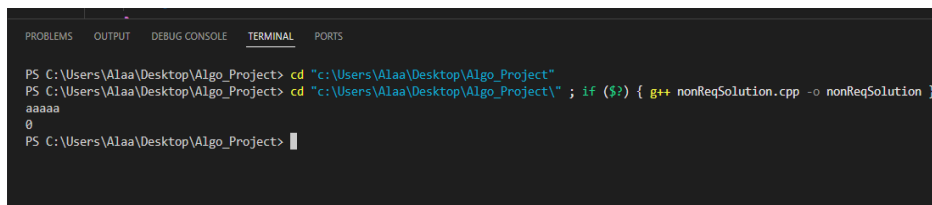


```
1 #include<iostream>
2 using namespace std;
3
4 int getLongestBalancedSubString(const string &s) {
5     int n = s.length(), longestBalancedSubString = 0;
6
7     for (int start = 0; start < n; start++) {
8         char c1 = 0, c2 = 0;
9         int countC1 = 0, countC2 = 0;
10
11         for (int end = start; end < n; end++) {
12             char currentChar = s[end];
13
14             if (currentChar == c1) countC1++;
15             else if (currentChar == c2) countC2++;
16             else if (c1 == 0) { c1 = currentChar; countC1++; }
17             else if (c2 == 0) { c2 = currentChar; countC2++; }
18             else break;
19
20             if (c1 && c2 && countC1 == countC2)
21                 longestBalancedSubString = max(longestBalancedSubString, end - start + 1);
22         }
23     }
24     return longestBalancedSubString;
25 }
26
27 int main() {
28     string s;
29     cin >> s;
30     cout << getLongestBalancedSubString(s) << endl;
31     return 0;
32 }
33
34
35
```

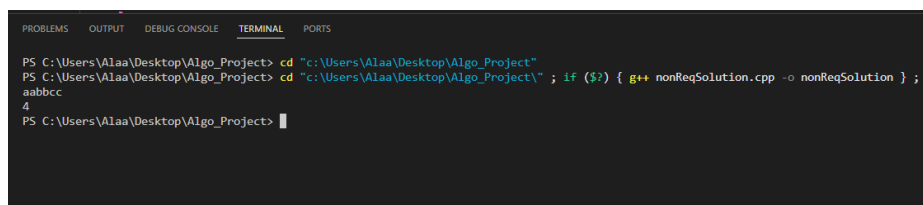
1.3 Non – Recursive Test



```
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project"
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ nonReqSolution.cpp -o nonReqSolution }
abababa
6
PS C:\Users\Alaa\Desktop\Algo_Project>
```



```
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project"
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ nonReqSolution.cpp -o nonReqSolution }
aaaaa
0
PS C:\Users\Alaa\Desktop\Algo_Project>
```



```
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project"
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ nonReqSolution.cpp -o nonReqSolution } ; i
aabbcc
4
PS C:\Users\Alaa\Desktop\Algo_Project>
```

1.4 Analysis & Complexity (Steps)

analysis of non recursive }

$$T(n) = \sum_{start=0}^{n-1} \sum_{end=start}^{n-1} 1$$

$\sum_{end=start}^{n-1} 1 = (n-1 - start + 1) = (n - start)$

Q 1e: $\sum_{i=0}^n 1 = (n - i + 1)$

$$T(n) = \sum_{start=0}^{n-1} (n - start) = \sum_{start=0}^{n-1} n - \sum_{start=0}^{n-1} start$$

[1] $\sum_{start=0}^{n-1} n = n \times n = n^2$ Rule: $\sum_{i=a}^b c = c(b-a+1)$
 $\hookrightarrow n(n-1-0+1) = n \times n = n^2$

[2] $\sum_{start=0}^{n-1} start = \frac{n(n-1)}{2}$ Rule: $0+1+2+\dots+(n-1) = \frac{n(n-1)}{2}$

$$T(n) = n^2 - \frac{n(n-1)}{2} = n^2 - \frac{n^2 - n}{2} = \frac{n^2 + n}{2}$$

$\approx O(n^2)$

The time complexity of non recursive algorithm = $O(n^2)$

property	Value
Time Complexity	$O(n^2)$
Space Complexity	$O(1)$
Best Case	$O(n)$ — 3rd char appears immediately
Worst Case	$O(n^2)$ — all chars are same or two chars

Second Algorithm — Recursive

2.1 Pseudo Code

```
1. algorithm IsBalanced(s, left, right)
2. {
3.   c1 := 0; c2 := 0; countC1 := 0; countC2 := 0;
4.   for i := l to r do {
5.     currentChar:= s[i];
6.     if currentChar = c1 then countC1 := countC1 + 1;
7.     else if currentChar = c1 then countC2 := countC2 + 1;;
8.     else if c1 = 0 then { c1 := currentChar; countC1 := 1; }
9.     else if c2 = 0 then { c2 := currentChar; countC2:= 1; }
10.    else return FALSE;
11.  }
12.  return (c1 != 0 AND c2 != 0 AND countC1 = countC2);
13. }

14. algorithm getLongestBalancedSubstring(s, start)
15. {
16.  n := length(s);
17.  if start = n then return 0;
18.  longestBalancedLength:= 0;
19.  for end := start to n - 1 do {
20.    if IsBalancedSubstring(s, start, end) = TRUE then
21.      if (end - start + 1) > longestBalancedLength then longestBalancedLength := end - start
+ 1;
22.  }
23.  return max(longestBalancedLength, getLongestBalancedSubstring (s, start + 1));
24. }

25. algorithm LongestBalancedSubstring(s)
26. {
27.  return getLongestBalancedSubstring (s, 0);
28. }

29. algorithm Main
30. {
31.  read s;
32.  write LongestBalancedSubstring(s);
33. }
```

2.2 Recursive Implementation (C++)

```
File Edit Selection View Go Run Terminal Help longestBalancedString
ReqSolution.cpp 1x nonRecursiveSolution.cpp 1x
3 using namespace std;
4
5 bool isBalancedSubstring(const string& s, int left, int right) {
6     char c1 = 0, c2 = 0;
7     int countC1 = 0, countC2 = 0;
8
9     for (int i = left; i <= right; i++) {
10         char currentChar = s[i];
11
12         if (currentChar == c1) countC1++;
13         else if (currentChar == c2) countC2++;
14         else if (c1 == 0) { c1 = currentChar; countC1++; }
15         else if (c2 == 0) { c2 = currentChar; countC2++; }
16         else return false;
17     }
18
19     return (c1 != 0 && c2 != 0 && countC1 == countC2);
20 }
21
22 int getLongestBalancedSubstring(const string& s, int start) {
23     int n = s.size();
24     if (start == n) return 0;
25
26     int longestBalancedLength = 0;
27
28     for (int end = start; end < n; end++) {
29         if (isBalancedSubString(s, start, end))
30             longestBalancedLength = max(longestBalancedLength, end - start + 1); // end - start + 1 current substring length
31     }
32
33     return max(longestBalancedLength, getLongestBalancedSubString(s, start + 1));
34 }
35
36 int longestBalancedSubString(const string& s) {
37     return getLongestBalancedSubString(s, 0);
38 }
39
40 int main() {
41     string s;
42     cin >> s;
43
44     cout << longestBalancedSubString(s) << endl;
45 }
```

2.3 Recursive Test

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Alaa\Desktop\Algo_Project>
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project"
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ ReqSolution.cpp -o ReqSolution } ; if ($?) { .\ReqSolution }
abababa
6
PS C:\Users\Alaa\Desktop\Algo_Project>
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project"
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ ReqSolution.cpp -o ReqSolution } ; if ($?) { .\ReqSolution }
cabbacc
4
PS C:\Users\Alaa\Desktop\Algo_Project>
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "c:\Users\Alaa\Desktop\Algo_Project"
2
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ ReqSolution.cpp -o ReqSolution } ; if ($?) { .\ReqSolution }
aaaaaaa
0
PS C:\Users\Alaa\Desktop\Algo_Project>
```

2.4 Analysis & Complexity (Steps)

Iteration method

3. iteration method

$$T(n) = T(n-1) + n^2$$
$$T(n-1) = T(n-2) + (n-1)^2$$
$$T(n-2) = T(n-3) + (n-2)^2$$
$$T(n-3) = T(n-4) + (n-3)^2$$
$$T(n) = T(n-2) + (n-1)^2 + n^2$$
$$T(n) = T(n-3) + (n-2)^2 + (n-1)^2 + n^2$$
$$T(n) = T(n-4) + (n-3)^2 + (n-2)^2 + (n-1)^2 + n^2$$
$$T(n) = T(n-k) + (n-(k-1))^2 + (n-(k-2))^2 + \dots + (n-1)^2 + n^2$$

$[n=k]$

$$T(n) = T(n-n) + (n-(n-1))^2 + (n-(n-2))^2 + \dots + (n-1)^2 + n^2$$
$$T(n) = T(0) + 1^2 + 2^2 + \dots + (n-1)^2 + n^2$$
$$T(n) = 1 + \sum_{i=1}^n i^2 = 1 + \frac{n(n+1)(2n+1)}{6}$$
$$T(n) = 1 + \frac{2n^3 + 3n^2 + n}{6} \rightarrow \approx O(n^3)$$

The time complexity of Recursive code
By Iteration method $= O(n^3)$

Master method

analysis of Recursive code

1) master method $\rightarrow$ decreasing "master"

$$T(n) = T(n-1) + n^2$$
$$a=1 \quad b=1 \quad P(n)=n^2$$
$$F(n) = O(n^k) \quad , \quad k \geq 0$$
$$F(n) = n^2 \quad \rightarrow \therefore k=2$$
$$T(n) = O(n^{k+1}) \rightarrow O(n^{2+1}) = O(n^3)$$

The time complexity of Recursive code $\rightarrow O(n^3)$

& iteration tree

2. Tree method

$$T(n) = T(n-1) + n^2$$

$T(n) \begin{cases} n=0 \\ T(n-1) + n^2 \quad n > 0 \end{cases}$

The analysis of Recursive code by Tree method
Time complexity = $O(n^3)$

$$1^2 + 2^2 + 3^2 + \dots + (n-1)^2 + n^2$$

$$= \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$= \frac{2n^3 + n^2 + 2n^2 + n}{6} = \frac{2n^3 + 3n^2 + n}{6}$$

$$\approx O(n^3)$$

Property	Value
Recurrence	$T(n) = T(n-1) + O(n^2)$
Time Complexity	$O(n^3)$
Space Complexity	$O(n)$ — call stack depth
Best Case	$O(n^2)$ — early breaks
Worst Case	$O(n^3)$ — no early breaks

Comparison — Iterative vs Recursive

Criterion	Iterative (Non-Recursive)	Recursive
Time Complexity	$O(n^2)$	$O(n^3)$
Space Complexity	$O(1)$	$O(n)$ — call stack
Stack Overflow	No risk	Risk on large inputs
Memory Usage	Lower	Higher
Performance	Faster	Slower
Best for	Production use	Conceptual understanding

Key Insight

Both algorithms solve the same problem correctly. The fundamental difference lies in how they re-use computed information:

Iterative: counts $c1$ and $c2$ incrementally as 'end' grows, never rescanning already-visited characters. This gives $O(n^2)$.

Recursive: at every call `RecursiveSolve(text, start, end)` it rescans the entire `[start..end]` range from scratch. This extra $O(n)$ scan per call elevates the total to $O(n^3)$.

Conclusion: the Iterative algorithm is strictly superior in both time and space for this problem.